Data-Driven Testing (DDT) in Pytest with Playwright

What is Data-Driven Testing?

Data-Driven Testing (DDT) is a testing approach where a single test case runs multiple times with different input data.
Instead of writing separate tests for each input, you **parameterize** the test with data sets.

In **Pytest**, this is achieved using the **@pytest.mark.parametrize** decorator.
In **Playwright**, the same logic is combined with browser automation to test UI workflows like login, search, or form submission.

Why Use Data-Driven Testing?

- Improves test coverage
- Reduces code duplication
- Makes tests maintainable
- Allows testers to separate **data** from **test logic**

Different Data Sources for DDT

Below are the common ways to provide test data to Pytest-Playwright tests.

1️⃣ Using Hardcoded Data (List or Tuple)

This is the simplest and fastest way to start data-driven testing.

Example:

```
import pytest
from playwright.sync_api import expect

# Test data defined in a list
test_data = [
    ("user1@example.com", "pass123", "valid"),
    ("wrong@example.com", "wrongpass", "invalid"),
]

@pytest.mark.parametrize("email, password, validity", test_data)
def test_login(page, email, password, validity):
    page.goto("https://demowebshop.tricentis.com/login")
    page.fill("#Email", email)
    page.fill("#Password", password)
    page.click('input[value="Log in"]')
    # Add assertions based on validity
```

📌 **Use Case:** Quick demo or when data is small and fixed.
📌 **Pros:** Very simple — no external files.
📌 **Cons:** Hard to scale; test data lives inside code.

2️⃣ Using JSON File

Data can be stored in a .json file and read dynamically.

Example:

data.json

```
[
  {"email": "valid@example.com", "password": "test123", "validity": "valid"},
  {"email": "wrong@example.com", "password": "abc123", "validity": "invalid"}
]
```

Test:

```
import json, pytest

with open("testdata/data.json") as f:
    login_data = json.load(f)

@pytest.mark.parametrize("email, password, validity", [
    (data["email"], data["password"], data["validity"]) for data in login_data
])
def test_login(page, email, password, validity):
    # Steps and assertions here
```

📌 **Use Case:** Structured and reusable test data.
📌 **Pros:** Easy to extend, machine-readable.
📌 **Cons:** Not suitable for tabular or large datasets.

3️⃣ Using CSV File

CSV files are great for tabular test data that manual testers can easily edit.

Example:

data.csv

```
email,password,validity
user1@example.com,pass123,valid
wrong@example.com,wrongpass,invalid
```

Test:

```
import csv, pytest

data = []
with open("testdata/data.csv") as file:
    reader = csv.DictReader(file)
    for row in reader:
        data.append((row["email"], row["password"], row["validity"]))

@pytest.mark.parametrize("email, password, validity", data)
def test_login(page, email, password, validity):
    # Steps and assertions here
```

📌 **Use Case:** Large tabular data.
📌 **Pros:** Editable via Excel or Google Sheets.
📌 **Cons:** All values are text; no nested data.

4️⃣ Using Excel File (openpyxl Library)

Excel is popular among testers for managing test cases and data.

Concept:

The **openpyxl** library reads Excel files row by row.
You can skip header rows and handle blank cells safely.

Example:

Excel file (data.xlsx):

email	password	validity
user1@example.com	pass123	valid
wrong@example.com	wrongpass	invalid

Test:

```
from openpyxl import load_workbook
import pytest

data = []
wb = load_workbook("testdata/data.xlsx")
sheet = wb.active

for row in sheet.iter_rows(min_row=2, values_only=True):
```

```
    email, password, validity = row
    data.append((str(email or ""), str(password or ""), str(validity or "")))

@pytest.mark.parametrize("email, password, validity", data)
def test_login(page, email, password, validity):
    # Steps and assertions here
```

📌 **Use Case:** When test data is managed in Excel sheets.
📌 **Pros:** Familiar to testers, organized layout.
📌 **Cons:** Requires openpyxl installation and handling of blank cells.

Common Test Flow in All Examples

1. **Load data** from a chosen source (List, JSON, CSV, Excel).
2. **Pass data** to test using @pytest.mark.parametrize.
3. **Perform actions** with Playwright (fill(), click(), etc.).
4. **Validate results** dynamically (e.g., success message, error message).

Summary Comparison

Source	Library	File Format	Ideal For	Advantages
List/Tuple	None	N/A	Small static data	Simple, fast setup
JSON	json	.json	Structured data	Flexible, readable
CSV	csv	.csv	Tabular data	Easy to maintain
Excel	openpyxl	.xlsx	Spreadsheet data	User-friendly for testers